239--Lab - Procesos administrativos

En esta práctica se muestra el funcionamiento de los siguientes comandos:

- *ps,* para mostrar los procesos en ejecución
- *top*, para mostrar el consumo de CPU, memoria RAM, el estado de las tareas y los procesos en ejecución
- *cron,* para programar tareas repetitivas

Crear una lista de procesos con *ps*

Obtenemos una lista de los procesos en ejecución (excepto los del usuario root) y los guardamos en un archivo, mediante el comando:

sudo ps -aux | grep -v root | sudo tee SharedFolders/processes.csv

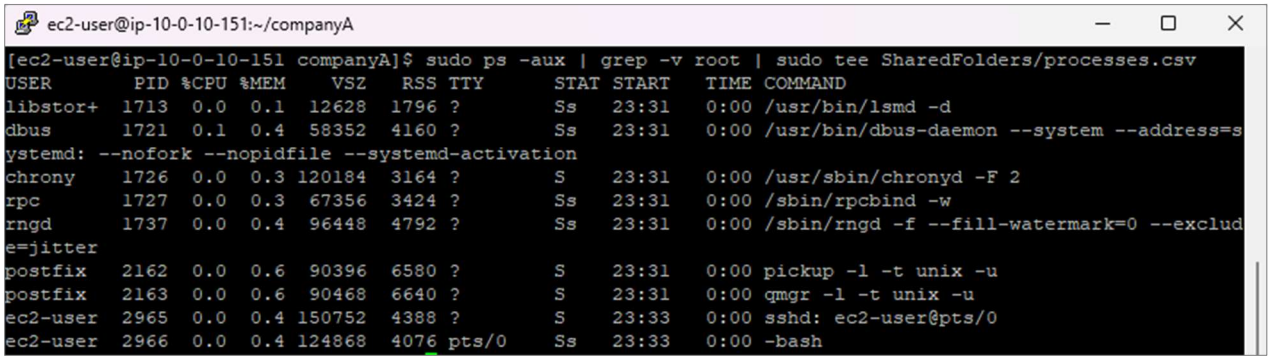
La opción *-a* muestra los procesos de todos los usuarios.

La opción *-u* muestra el formato orientado al usuario.

La opción -x incluye también los servicios del sistema o daemons.

Con la opción *-v* del comando *grep* indicamos que se exceptúe el texto "root".

Con el comando *tee* indicamos que el resultado se escriba en el archivo processes.csv y también se imprima en pantalla.



```
ec2-user@ip-10-0-10-151:~/companyA
[ec2-user@ip-10-0-10-151 companyA]$ sudo ps -aux | grep -v root | sudo tee SharedFolders/processes.csv
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
libstor+  1713  0.0  0.1  12628  1796 ?        Ss   23:31   0:00 /usr/bin/lsmd -d
dbus      1721  0.1  0.4  58352  4160 ?        Ss   23:31   0:00 /usr/bin/dbus-daemon --system --address=s
ystemd: --nofork --nopidfile --systemd-activation
chrony    1726  0.0  0.3 120184  3164 ?        S    23:31   0:00 /usr/sbin/chronyd -F 2
rpc       1727  0.0  0.3  67356  3424 ?        Ss   23:31   0:00 /sbin/rpcbind -w
rngd      1737  0.0  0.4  96448  4792 ?        Ss   23:31   0:00 /sbin/rngd -f --fill-watermark=0 --exclud
e=jitter
postfix   2162  0.0  0.6  90396  6580 ?        S    23:31   0:00 pickup -l -t unix -u
postfix   2163  0.0  0.6  90468  6640 ?        S    23:31   0:00 qmgr -l -t unix -u
ec2-user  2965  0.0  0.4 150752  4388 ?        S    23:33   0:00 sshd: ec2-user@pts/0
ec2-user  2966  0.0  0.4 124868  4076 pts/0    Ss   23:33   0:00 -bash
```

Verificamos el contenido del archivo.

```
ec2-user@ip-10-0-10-151:~/companyA
[ec2-user@ip-10-0-10-151 companyA]$ cat SharedFolders/processes.csv
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
libstor+  1713  0.0  0.1  12628  1796 ?        Ss   23:31   0:00 /usr/bin/lsmd -d
dbus      1721  0.1  0.4  58352  4160 ?        Ss   23:31   0:00 /usr/bin/dbus-daemon --system --address=s
ystemd: --nofork --nopidfile --systemd-activation
chrony    1726  0.0  0.3 120184  3164 ?        S    23:31   0:00 /usr/sbin/chronyd -F 2
rpc       1727  0.0  0.3  67356  3424 ?        Ss   23:31   0:00 /sbin/rpcbind -w
rngd      1737  0.0  0.4  96448  4792 ?        Ss   23:31   0:00 /sbin/rngd -f --fill-watermark=0 --exclud
e=jitter
postfix   2162  0.0  0.6  90396  6580 ?        S    23:31   0:00 pickup -l -t unix -u
postfix   2163  0.0  0.6  90468  6640 ?        S    23:31   0:00 qmgr -l -t unix -u
ec2-user  2965  0.0  0.4 150752  4388 ?        S    23:33   0:00 sshd: ec2-user@pts/0
ec2-user  2966  0.0  0.4 124868  4076 pts/0    Ss   23:33   0:00 -bash
```

Crear una lista de procesos con *top*

El comando *top* sirve principalmente para vigilar el rendimiento del sistema y saber qué programas están consumiendo más recursos.

En la sección *Tasks* podemos ver el número de tareas en ejecución, suspendidas, detenidas y zombie.

En la sección *%Cpu(s)* podemos ver el tiempo que el sistema ha dedicado a diversas tareas.

También se muestra el uso de memoria del sistema y el espacio de intercambio.

```
ec2-user@ip-10-0-10-151:~/companyA
[ec2-user@ip-10-0-10-151 companyA]$ top
top - 23:37:27 up 6 min,  1 user,  load average: 0.00, 0.03, 0.01
Tasks:  87 total,   1 running,  47 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
KiB Mem :   962712 total,   374552 free,    73456 used,   514704 buff/cache
KiB Swap:        0 total,        0 free,        0 used.   746652 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU %MEM     TIME+ COMMAND
    1 root      20   0  189048   5384   3916 S   0.0  0.6   0:01.62 systemd
    2 root      20   0       0      0      0 S   0.0  0.0   0:00.00 kthreadd
    4 root       0 -20       0      0      0 I   0.0  0.0   0:00.00 kworker/0:0H
    5 root      20   0       0      0      0 I   0.0  0.0   0:00.13 kworker/u4:0
    6 root       0 -20       0      0      0 I   0.0  0.0   0:00.00 mm_percpu_wq
    7 root      20   0       0      0      0 S   0.0  0.0   0:00.05 ksoftirqd/0
    8 root      20   0       0      0      0 I   0.0  0.0   0:00.09 rcu_sched
    9 root      20   0       0      0      0 I   0.0  0.0   0:00.00 rcu_bh
   10 root      rt   0       0      0      0 S   0.0  0.0   0:00.00 migration/0
   11 root      rt   0       0      0      0 S   0.0  0.0   0:00.00 watchdog/0
   12 root      20   0       0      0      0 S   0.0  0.0   0:00.00 cpuhp/0
   13 root      20   0       0      0      0 S   0.0  0.0   0:00.00 cpuhp/1
```

Para salir de *top* tecleamos q y Enter.

Con el comando *top -hv* podemos ver información del programa mismo.

La opción *-h* muestra el menú de ayuda de *top*.

La opción -v muestra la versión específica del paquete procps-ng (el software que contiene a top) que está instalado en el sistema.

```
ec2-user@ip-10-0-10-151:~/companyA
[ec2-user@ip-10-0-10-151 companyA]$ top -hv
  procps-ng version 3.3.10
Usage:
  top -hv | -bcHiOSs -d secs -n max -u|U user -p pid(s) -o field -w [cols]
```

Programar una tarea con *Cron*

Cron es un comando que permite programar la ejecución repetitiva de tareas.

Requerimos crear un archivo con las instrucciones que serán ejecutadas. Para esto usamos el comando *sudo crontab -e*.

```
[ec2-user@ip-10-0-10-151 companyA]$ sudo crontab -e
```

Se abre el editor de texto.

Tecleamos i para entrar al modo de inserción y escribimos las siguientes instrucciones:

SHELL=/bin/bash

PATH=/usr/bin:/bin:/usr/local/bin

MAILTO=root

*0 * * * * ls -la $(find .) | sed -e 's/..csv/#####.csv/g' > /home/ec2-user/companyA/SharedFolders/filteredAudit.csv*

```
ec2-user@ip-10-0-10-151:~/companyA
SHELL=/bin/bash
PATH=/usr/bin:/bin:/usr/local/bin
MAILTO=root
0 * * * * ls -la $(find .) | sed -e 's/..csv/#####.csv/g' > /home/ec2-user/companyA/SharedFolders/filtered
Audit.csv
~
~
~
~
~
~
~
~
~
~
```

Para guardar los cambios y salir del editor tecleamos :wq y Enter.

El comando *$(find .)* busca todos los archivos y carpetas a partir del directorio actual (.) y hacia abajo (subcarpetas). Con el comando ls -la generamos una lista.

En el resultado de esta lista de archivos reemplazamos todos los "..csv" por "#####.csv".

El resultado de este reemplazo lo guardamos en el archivo filteredAudit.csv

La línea *SHELL=/bin/bash* le indica al sistema qué intérprete debe usar para ejecutar los comandos programados.

La línea *PATH=/usr/bin:/bin:/usr/local/bin* le indica al sistema en qué carpetas debe buscar los archivos ejecutables.

La línea *MAILTO=root* sirve para definir a quién debe enviar el sistema los resultados que generen las tareas programadas.

Con *0* * * * * indicamos que la tarea se ejecutará una vez por hora, todos los días.

El archivo crontab incluye 5 campos que indican cuando se ejecutará la tarea:

```
0  * * * *
|  |  |  |  |
|  |  |  |  +----- Día de la semana (0-6) -> * (Todos los días)
|  |  |  +-------- Mes (1-12) -> * (Todos los meses)
|  |  +----------- Día del mes (1-31) -> * (Todos los días)
|  +-------------- Hora (0-23) -> * (A cada hora)
+----------------- Minuto (0-59) -> 0 (Solo en el minuto 0)
```

Si quisiéramos ejecutarlo a cada minuto sería * * * * *

Con el comando *sudo crontab -l* verificamos las tareas programadas.

```
ec2-user@ip-10-0-10-151:~/companyA
[ec2-user@ip-10-0-10-151 companyA]$ sudo crontab -l
SHELL=/bin/bash
PATH=/usr/bin:/bin:/usr/local/bin
MAILTO=root
0 * * * * ls -la $(find .) | sed -e 's/..csv/#####.csv/g' > /home/ec2-user/companyA/SharedFolders/filtered
Audit.csv
```